

UNIVERSIDAD NACIONAL DE MAR DEL PLATA

Facultad de Ingeniería

Trabajo final

**Desarrollo de un Analizador Vectorial de Redes para
medición no invasiva de glucemia**

Victoria Carla Torres



Ingeniería Electrónica
Departamento de Ingeniería Electrónica y Computación
Septiembre 2026

Agradecimientos

Índice

Agradecimientos	1
Siglas y abreviaturas	4
Resumen	4
1 Introducción	6
1.1 Contexto y motivación	6
1.2 Partes interesadas	7
1.3 Objetivos	7
1.4 Organización del informe	7
2 Marco teórico	8
2.1 Analizador Vectorial de Redes	8
2.2 Parámetros de dispersión (S)	8
2.2.1 Parámetro S11	9
2.2.2 Calibración S11	9
2.2.2.1 Errores	9
2.2.2.2 Método SOL	9
2.3 Acoplador Direccional	10
2.3.1 Puente resistivo	11
2.4 Buenas prácticas de diseño en RF	11
2.5 Heterodinación de señales	12
2.6 Procesamiento digital de señales	13
2.6.1 Filtros digitales	13
2.6.1.1 Diseño de filtros FIR mediante el método del inventariado	14
2.6.2 Transformada Rápida de Fourier	14
3 Proyecto	16
3.1 Descripción general del producto	16
3.2 Hardware	16
3.2.1 Generadores de RF	17
3.2.2 Implementación en PCB	17
3.3 Firmware	17
3.4 Software	17
3.4.1 Estimación del S11	17
3.5 Interfaz de usuario	18
4 Resultados	20
5 Conclusiones y trabajos futuros	21

6	Esquemáticos y código	22
6.1	Esquemáticos Altium	22
6.2	Interfaz de usuario	22
6.2.1	Core	22
6.2.2	Graphical User Interface	29
	Apéndices	42
	Bibliografía	44

Siglas y abreviaturas

Nombre	Descripción
VNA	Analizador Vectorial de Redes
UHF	Ultra High Frequency
LAC	Laboratorio de Comunicaciones
ICYTE	Instituto de Investigaciones Científicas y Tecnológicas en Electrónica
FI	Frecuencia Intermedia
DUT	Device Under Test
RF	Radiofrecuencia
PCB	Printed Circuit Board
OL	Oscilador Local
FIR	Finite Impulse Response
FFT	Fast Fourier Transform
IC	Integrated Circuits
IA	Inteligencia Artificial

Resumen

El proyecto propone el diseño, implementación, calibración y ensayo de un Analizador Vectorial de Redes (VNA) de un puerto, que opere en una banda de frecuencias UHF (Ultra High Frequency) que va desde los 1,8 hasta los 2,2 GHz, adaptado para operar en conjunto con un sensor de glucemia no invasivo.

El principio de medición se basa en la existencia de una correlación entre la concentración de glucosa en sangre y sus propiedades eléctricas (permitividad relativa y conductividad) [15], por lo que es razonable suponer que las variaciones en la composición dieléctrica de una muestra de tejido biológico pueden alterar la distribución de los campos electromagnéticos presentes en un sensor de microondas.

Capítulo 1

Introducción

1.1 Contexto y motivación

La diabetes es una afección crónica que se desencadena cuando el organismo pierde su capacidad de producir suficiente insulina o de utilizarla con eficacia para mantener los niveles de glucosa en la sangre dentro de los márgenes normales. La cantidad de adultos que la padecen se ha casi cuadruplicado entre 1980 y 2016, con un aumento en la prevalencia más acelerado en países de ingresos bajos y medios que en países de ingresos altos [12]. Para 2021 fue la causa directa de 1,6 millones de muertes [11]. Existen numerosos dispositivos que permiten realizar un control de la glucosa en sangre pero requieren, en general, de una punción en alguna parte del cuerpo para extraer una muestra de sangre. El mecanismo tradicional requiere depositar la muestra en una tira reactiva dentro de un medidor para obtener el valor de glucemia. Se recomienda, en promedio, que los pacientes realicen 4 mediciones al día (dependiendo de la estabilidad de los niveles o del régimen de tratamiento). Dado que la extracción suele ser dolorosa, además de que tiene un costo asociado al gasto de cada tira reactiva, la mayoría de los pacientes no logran tener un buen monitoreo de su glucemia. Por este motivo se ha incursionado en la glucometría semi-invasiva como los sistemas CGM (Continuous Glucose Monitoring) [1] que obtienen mediciones del líquido intersticial y requieren la colocación de un sensor debajo de la piel que se debe reemplazar periódicamente, por lo que siguen siendo invasivos, incómodos de usar y descartables. El estudio de propuestas para la medición no invasiva ha avanzado en los últimos años, pero aún no existe una técnica ampliamente adoptada ni dispositivos comerciales fiables. Se ha observado que existe una correlación entre la concentración de glucosa en sangre y sus propiedades eléctricas (permitividad relativa y conductividad) [15], por lo que es razonable suponer que las variaciones en la composición dieléctrica de una muestra de tejido biológico pueden alterar la distribución de los campos electromagnéticos presentes en un sensor de microondas.

Por esta razón el Laboratorio de Comunicaciones (LAC), perteneciente al Instituto de Investigaciones Científicas y Tecnológicas en Electrónica (ICYTE), que depende de la UNMDP y del Consejo Nacional de Investigaciones Científicas y Técnicas (CONICET) tiene una línea de investigación cuyo objetivo es diseñar e implementar un prototipo que permita realizar una medición de glucemia, basado en un VNA y un sensor de microondas.

Un Analizador Vectorial de Redes es un instrumento de medición que permite caracterizar sistemas en alta frecuencia. La forma en que lo hace es haciendo incidir una señal de microondas en los puertos del sistema, y midiendo las señales que salen del resto de los puertos.

El Laboratorio busca optimizar, compactar y mejorar el rendimiento del VNA actual, por lo que requiere el diseño y desarrollo de un VNA de un puerto que esté adaptado a un sensor de microondas que resuena alrededor de los 2 GHz.

1.2 Partes interesadas

El Grupo de Investigación del LAC es quien solicita el desarrollo. Por esta razón, el equipo se utilizará bajo su supervisión, y sus posibles usuarios son:

- Ingenieros, técnicos y/o radioaficionados;
- Estudiantes de ingeniería.

1.3 Objetivos

El objetivo principal del proyecto es el diseño, desarrollo, calibración y ensayo de un VNA de bajo costo y tamaño compacto, que opere en la banda de frecuencias de UHF que abarca desde 1,8 a 2,2 GHz, orientado a la medición no invasiva de glucemia.

Los objetivos particulares son:

- Implementar en una placa de circuito impreso el VNA;
- Generar un barrido en la banda de frecuencias de interés;
- Implementar un sistema heterodino que permita trabajar con señales de frecuencia intermedia (FI) en la banda de audio;
- Digitalizar las señales obtenidas para aplicar un tratamiento digital que permita calcular el coeficiente de reflexión del sistema bajo prueba (DUT - *Device Under Test*);
- Implementar una rutina de calibración para eliminar errores sistemáticos del sistema;

1.4 Organización del informe

El informe se organiza de la siguiente manera: en el capítulo primero se introduce el proyecto y se brinda un punteo de los objetivos más importantes. El capítulo 2 se centra en el marco teórico, donde se explican las bases para entender el funcionamiento del dispositivo. El capítulo 3 describe el proyecto con su implementación. En el capítulo 4 se muestran los resultados y mediciones. El capítulo 5 se dedica a conclusiones y trabajos futuros, y el 6 exhibe los esquemáticos y códigos utilizados. Por último se anexan los apéndices en donde se encuentran: el estudio preliminar, el plan de trabajo, las especificaciones de requerimientos, funcional y técnica y el plan de pruebas.

Capítulo 2

Marco teórico

2.1 Analizador Vectorial de Redes

El Analizador Vectorial de Redes fue definido en sus inicios como un instrumento que permite medir los parámetros de redes eléctricas [6]. En su esquema más básico posee una fuente de radiofrecuencias (RF), que genera una señal de estímulo para el dispositivo bajo prueba, y un banco de receptores que miden las señales que salen del DUT y las comparan con la onda incidente. Los resultados se procesan y muestran al usuario por medio de alguna interfaz.

Dependiendo de los dispositivos que se buscan caracterizar, se determina la cantidad de puertos que debe tener el VNA. Un puerto permite medir el parámetro de reflexión. Con dos puertos, se puede medir además el de transmisión. Si posee un solo canal, se puede medir la transmisión inversa y la reflexión en la salida invirtiendo físicamente el dispositivo bajo prueba. Este paso se puede evitar agregando un canal, lo que incrementa el costo del VNA.

2.2 Parámetros de dispersión (S)

Los parámetros de dispersión, también llamados S por su nombre en inglés (*Scattering*) permiten caracterizar redes de dos o más puertos en función de sus ondas entrantes y salientes. Se especifican para una impedancia de referencia y a una frecuencia específica, y son útiles en altas frecuencias porque otros tipos de parámetros (como los Z o los Y) no pueden ser medidos directamente.

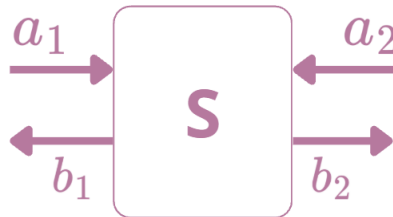


Figura 2.1: Parámetros S en una red de dos puertos

Según las ondas incidentes a_1 y a_2 y las reflejadas b_1 y b_2 , los parámetros S se definen como:

$$S_{11} = \left. \frac{b_1}{a_1} \right|_{a_2=0}; S_{12} = \left. \frac{b_1}{a_2} \right|_{a_1=0}; S_{21} = \left. \frac{b_2}{a_1} \right|_{a_2=0}; S_{22} = \left. \frac{b_2}{a_2} \right|_{a_1=0}$$

Para lograr que las a sean iguales a 0, el puerto al que pertenecen debe estar adaptado, garantizando que no haya ninguna señal incidente.

2.2.1 Parámetro S11

El parámetro S_{11} se conoce como el coeficiente de reflexión. Se puede definir en función de las ondas incidente y reflejada, o en función de la impedancia característica del sistema Z_0 y la impedancia de carga (es decir, del sistema bajo prueba):

$$S_{11}(DUT) = \Gamma = \frac{Z_L - Z_0}{Z_L + Z_0} \quad (2.2.1)$$

Expresado en dB, se expresa:

$$S_{11}(DUT)[dB] = 20 \cdot \log(|\Gamma|) \quad (2.2.2)$$

2.2.2 Calibración S11

2.2.2.1 Errores

Los errores que pueden aparecer en una medición se pueden dividir en dos grandes clases: los errores sistemáticos y los aleatorios. Los primeros se pueden corregir, mientras que los segundos se encuentran en toda medición y están fuera del control del usuario.

En el caso de dispositivos electrónicos algunos errores sistemáticos son los producidos por variaciones en la potencia de salida de los generadores, pérdidas de inserción en materiales conductores y conectores, desfasajes ocasionados por diferencias en longitudes eléctricas o retardos en los circuitos integrados, reflexiones generadas por desadaptación de impedancias y deriva térmica. Algunos errores aleatorios pueden ser ruido térmico, de cuantización e interferencia electromagnética.

La calibración en un VNA permite corregir los errores sistemáticos, excluyendo: comportamientos alineales; repetitividad en conectores; errores en las cargas de calibración; y deriva térmica. Este último se puede corregir solo si la calibración se realiza inmediatamente antes de la medición.

2.2.2.2 Método SOL

En VNAs de un puerto se pueden simplificar los errores a un modelo de tres términos que describen la directividad, adaptación del puerto y errores de seguimiento (variaciones de amplitud y fase en función de la frecuencia)[13]. Usando tres cargas distintas de valores conocidos, se puede formar un sistema de tres ecuaciones con tres incógnitas y resolver para obtener los términos del error:

$$e_{00} + \Gamma_1 \Gamma_{M1} e_{11} - \Gamma_1 \Delta_e = \Gamma_{M1} \quad (2.2.3a)$$

$$e_{00} + \Gamma_2 \Gamma_{M2} e_{11} - \Gamma_2 \Delta_e = \Gamma_{M2} \quad (2.2.3b)$$

$$e_{00} + \Gamma_3 \Gamma_{M3} e_{11} - \Gamma_3 \Delta_e = \Gamma_{M3} \quad (2.2.3c)$$

Donde Γ_M es el parámetro de reflexión medido para cada carga, e_{00} es la directividad, e_{11} la adaptación del puerto, $e_{10}e_{01}$ el error de seguimiento (*tracking*) y $\Delta_e = e_{00}e_{11} - (e_{10}e_{01})$. Los términos del error se pueden observar en la figura 2.2.

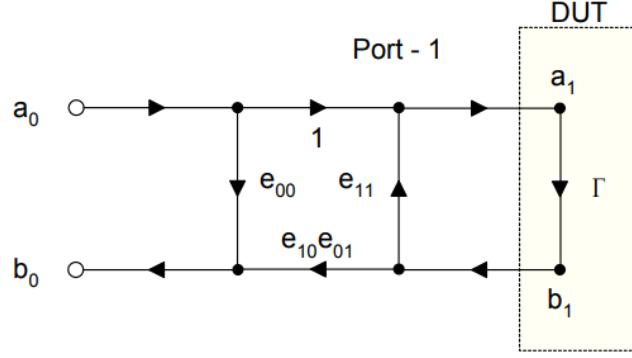


Figura 2.2: Términos de error para un VNA de un puerto

Si las cargas independientes son un circuito abierto, cortocircuito y carga de 50Ω entonces el método de calibración se denomina SOL (Short, Open, Load) y las ecuaciones (2.2.3) se simplifican a:

$$e_{00} - \Gamma_{MS}e_{11} + \Delta_e = \Gamma_{MS} \quad (2.2.4a)$$

$$e_{00} + \Gamma_{MO}e_{11} - \Delta_e = \Gamma_{MO} \quad (2.2.4b)$$

$$e_{00} = \Gamma_{ML} \quad (2.2.4c)$$

Luego el Γ real se calcula como:

$$\Gamma = \frac{\Gamma_M - e_{00}}{\Gamma_M e_{11} - \Delta_e} \quad (2.2.5)$$

2.3 Acoplador Direccional

En la mayoría de diseños en electrónica, el acoplamiento electromagnético entre líneas de transmisión se considera indeseable porque interfiere con la señal que se busca transmitir. Una excepción es el caso de acopladores, donde el acoplamiento es un parámetro de diseño que permite medir la señal de una línea con un alto aislamiento, es decir, sin interrumpir su paso. Además se pueden diseñar un par de líneas acopladas que verifiquen que en uno de sus puertos la señal incidente se anula, generando un punto que permite medir la onda reflejada.

Las principales propiedades que caracterizan un acoplador son la directividad, el aislamiento, el acoplamiento y la pérdida de inserción [14]. La directividad mide la capacidad de separar las ondas que se propagan en distintas direcciones. El aislamiento, la potencia que llega al puerto desacoplado. El acoplamiento indica la proporción de potencia que se entrega al puerto acoplado. La pérdida de inserción muestra cuánta potencia (que se debería transmitir) se pierde por ser entregada a los puertos acoplados y aislados, y las pérdidas en los materiales (dieléctrico y resistividad de las pistas).

Un acoplador ideal es un dispositivo pasivo, que posee directividad y aislamiento infinitos.

Existen acopladores direccionales como el HP 778d [7] que posee una directividad mayor a 36dB y pérdidas por inserción menores a 0,6dB, pero tiene un ancho de banda que permite hacer mediciones entre 100 MHz y 2 GHz, y sus dimensiones físicas y peso hacen que su utilidad se limite a mediciones en laboratorio. Para soluciones integradas hay alternativas comerciales como el ZABDC20-322 de Minicircuits [9] pero su costo hace que el diseño no sea escalable.

2.3.1 Puente resistivo

Una solución compacta, económica y con un ancho de banda teórico infinito es el puente resistivo tipo Wheatstone. Como se puede observar en la figura 2.3 es una red simple de comparación de impedancias, formada por cuatro resistencias sobre las cuales se inyecta una señal de referencia en la parte superior. Una de las resistencias es variable (Z_x), mientras que el resto son iguales entre sí. Se utiliza un voltímetro para medir la diferencia de potencial entre las ramas. Si la resistencia variable es de 50Ω , el puente queda balanceado y el voltímetro indica un cero. Si es menor a 50Ω , la rama izquierda presenta un voltaje menor a la derecha, mientras que para valores mayores ocurre lo opuesto. Si la señal incidente es sinusoidal es posible además caracterizar cargas reactivas midiendo el desfase de la señal Z_x respecto de la incidente. Este es el principio que permite al puente comportarse como un acoplador direccional: la diferencia de potencial entre las ramas es función de la onda reflejada equivalente.

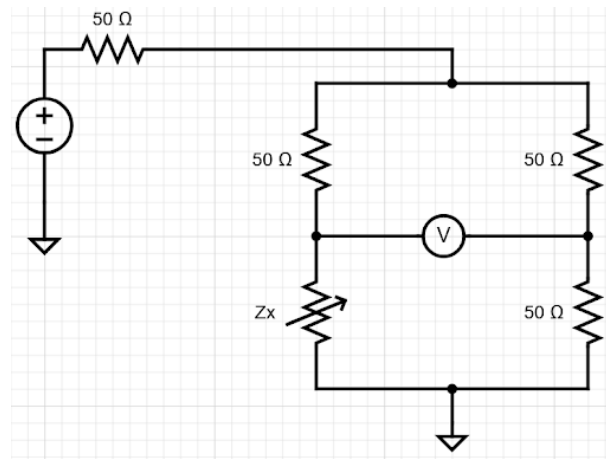


Figura 2.3: Esquema básico de un puente de Wheatstone

Para una aplicación de reflectometría, la magnitud de la impedancia de entrada al dispositivo bajo prueba haría las veces del Z_x a caracterizar en este esquema simplificado. Esto, además de la capacidad de lograr directividad y ancho de banda elevados, lo vuelve una alternativa muy conveniente a la hora de implementar un VNA que funcione con buena sensibilidad en un rango amplio de frecuencias.

2.4 Buenas prácticas de diseño en RF

Al momento de rutear un PBC (*Printed Circuit Board* - Placa de Circuito Impreso) se deben tener en consideración algunas buenas prácticas que ayudan a mantener la integridad de las señales, la impedancia de las líneas y evitar la interferencia electromagnética, sobre todo cuando se trabaja en RF.

Se debe garantizar que las pistas cumplan con la impedancia requerida de 50Ω . Esto implica calcular el ancho de la línea usando las ecuaciones de diseño según el tipo de línea y las características de los materiales a utilizar (generalmente brindadas por el fabricante de PCB). Se pueden encontrar calculadoras online o integradas en los software de diseño con este fin. Se deben evitar ángulos rectos, que causan discontinuidades y aumentan la capacitancia parásita.

El plano de masa que se encuentra debajo de las pistas de RF debe ser continuo, para que la corriente de retorno pueda seguir el camino de menor inductancia. Por esta razón no se recomienda rutear en el plano inferior al de las rutas de RF, pero si se necesita hacerlo no se deben atravesar las regiones donde en la capa superior haya trazos de RF.

Se deben usar las técnicas *vía stitching* y *vía shielding*. La segunda se usa para reducir la interferencia electromagnética y el crosstalk [3], mientras que la primera "ata" los planos de masa

de distintas capas, ayudando a mantener baja impedancia y caminos de retorno cortos. Además evita resonancias dentro de la placa.

Los capacitores de desacople se deben ubicar lo más cerca posible de los integrados, de forma de poder suministrar rápidamente los picos de corriente que demandan los integrados que la fuente de alimentación no puede entregar [2]. También deben utilizarse cuando se suministra alimentación a un trazo a través de vías.

Los trazos deben ser lo más cortos posible, para minimizar la atenuación de las señales y reflexiones indeseadas.

2.5 Heterodinación de señales

Las señales que se intentan medir en un VNA son de naturaleza analógica, por lo que para realizar el cálculo del parámetro de reflexión o S11 primero se debe poder medir su amplitud y fase. Existen integrados que pueden obtener la magnitud de una onda de RF como el HMC601 [5] pero se pierde información de la fase. Por eso se busca una forma de bajar la frecuencia de la señal para poder adquirirla de forma digital.

Un esquema heterodino se basa en la combinación de una señal de RF de interés con otra señal generada localmente por un oscilador local (OL o LO por sus siglas en inglés) desplazada en frecuencia un valor constante, llamado frecuencia intermedia (FI).

Cuando se multiplican idealmente en el dominio del tiempo dos señales sinusoidales $v_{RF}(t) = A_{RF} \cos(2\pi f_{RF}t)$ y $v_{LO}(t) = A_{LO} \cos(2\pi(f_{RF} + f_{IF})t)$ (suponiendo que tienen la misma fase inicial), a la salida existen componentes espectrales en la suma y diferencia de las frecuencias originales:

$$v_{mix}(t) = k \cdot v_{RF}(t) \cdot v_{LO}(t) \quad (2.5.1a)$$

$$v_{mix}(t) = k A_{RF} A_{LO} \cos(2\pi f_{RF}t) \cos(2\pi(f_{RF} + f_{IF})t) \quad (2.5.1b)$$

$$(2.5.1c)$$

Reescribiendo con la identidad trigonométrica $\cos(\alpha) \cos(\beta) = \frac{1}{2} [\cos(\alpha - \beta) + \cos(\alpha + \beta)]$ entonces:

$$v_{mix}(t) = \frac{k A_{RF} A_{LO}}{2} \left[\underbrace{\cos(2\pi f_{FI}t)}_{\text{Componente Resta (FI)}} + \underbrace{\cos(2\pi(2f_{RF} + f_{FI})t)}_{\text{Componente Suma (Alta Frecuencia)}} \right] \quad (2.5.2a)$$

Mediante un filtrado se puede aislar la componente diferencia, logrando así trasladar la información de la señal de RF hacia la FI:

$$v_{IF}(t) = \frac{k A_{RF} A_{LO}}{2} \cos(2\pi f_{IF}t) \quad (2.5.3)$$

Utilizando un mezclador doble se pueden heterodinar dos señales con un solo oscilador local, como muestra la figura 2.4 . De esta forma obtienen las señales incidente y reflejada, conservando su amplitud y fase, en una frecuencia de audio que facilite su adquisición y procesamiento.

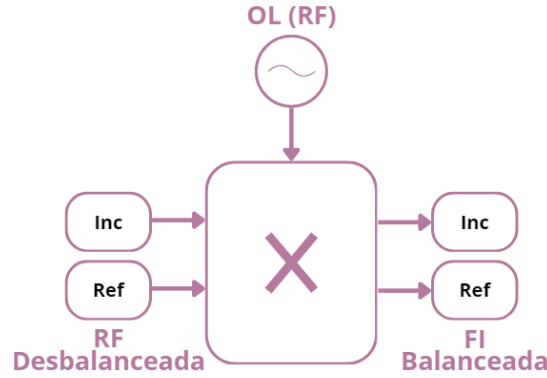


Figura 2.4: Heterodinación de señales

2.6 Procesamiento digital de señales

Para poder aplicar un procesamiento digital cualquier señal analógica continua en el tiempo debe ser muestreada, es decir convertirla en una secuencia de números de precisión finita. El proceso implica una pérdida de información de los intervalos intermedios, pero según el teorema de Nyquist-Shannon una señal de banda limitada puede reconstruirse inequívocamente a partir de sus muestras si la frecuencia de muestreo f_s cumple la condición $f_s \geq 2f_{\max}$, donde $f_{\max}$ representa la componente frecuencial de mayor valor de la señal [8].

Una vez que la señal está digitalizada, para poder extraer su amplitud y fase, sabiendo que se encuentra a una frecuencia de valor FI, se debe aplicar un filtrado digital y realizar la Transformada Rápida de Fourier (FFT - *Fast Fourier Transform*)

2.6.1 Filtros digitales

El objetivo principal de la etapa de filtrado es aislar la señal de FI eliminando el ruido de banda ancha, las interferencias de baja frecuencia y armónicos de orden superior.

La señal muestreada $x[n]$ pasa por un filtro pasabanda digital caracterizado por una respuesta al impulso $h[n]$. La operación se define mediante la convolución discreta:

$$y[n] = x[n] * h[n] = \sum_{k=0}^M h[k] x[n - k] \quad (2.6.1)$$

donde M representa el orden del filtro. Los coeficientes $h[k]$ se diseñan para configurar una banda de paso en torno a la frecuencia de FI.

Los filtros digitales se pueden clasificar en dos grandes familias según la naturaleza de su respuesta al impulso: filtros de respuesta finita (*Finite Impulse Response*, FIR) y filtros de respuesta infinita (*Infinite Impulse Response*, IIR).

Un filtro IIR se caracteriza por una respuesta al impulso de duración infinita, que se logra mediante una estructura recursiva con realimentación de las salidas anteriores. Su ecuación en diferencias se expresa como:

$$y[n] = \sum_{k=0}^M b_k x[n - k] + \sum_{k=1}^N a_k y[n - k] \quad (2.6.2)$$

La presencia de los coeficientes a_k introduce polos en el plano z fuera del origen. Si bien permiten lograr bandas de transición abruptas con órdenes bajos, presentan dos desventajas fundamentales: no garantizan fase lineal en toda la banda de paso y pueden ser inestables si sus polos migran fuera del círculo unitario debido a cuantización de precisión finita.

Por otro lado, un filtro FIR presenta una respuesta al impulso de duración finita N_{taps} (comúnmente denominada número de coeficientes o *taps*), siendo $h[n] = 0$ para $n < 0$ y $n \geq N_{\text{taps}}$. Su ecuación en diferencias es puramente no recursiva:

$$y[n] = \sum_{k=0}^M b_k x[n-k] = \sum_{k=0}^{N_{\text{taps}}-1} h[k] x[n-k] \quad (2.6.3)$$

donde $M = N_{\text{taps}} - 1$ representa el orden del filtro. Su transferencia $H(z)$ es causal, estable y realizable. Todos sus polos se ubican de manera trivial en el origen ($z = 0$), lo que garantiza estabilidad incondicional.

Si la secuencia de coeficientes $h[n]$ cumple la condición de simetría par alrededor de su centro ($h[n] = h[N_{\text{taps}} - 1 - n]$), la respuesta en frecuencia presenta la forma:

$$H(e^{j\omega}) = |H(e^{j\omega})| e^{-j\omega\tau_g} \quad (2.6.4)$$

donde el retardo de grupo τ_g es estrictamente constante para todas las frecuencias:

$$\tau_g = \frac{N_{\text{taps}} - 1}{2} \quad [\text{muestras}] \quad (2.6.5)$$

Un retardo de grupo constante asegura que el filtro no introduce distorsión de fase no lineal, preservando la diferencia de fase relativa entre el canal de referencia (V_{ref}) y el de medición (V_{med}).

2.6.1.1 Diseño de filtros FIR mediante el método del enventanado

La forma más simple de diseño de filtros FIR es el llamado método del enventanado (*window design method*) [10]. Este método consiste en truncar y desplazar la respuesta al impulso ideal de duración infinita, $h_d[n]$ (correspondiente a un filtro pasabanda ideal centrado en f_{FI}), multiplicándola punto a punto por una función ventana finita $w[n]$ de longitud N_{taps} :

$$h[n] = h_d[n] \cdot w[n], \quad 0 \leq n \leq N_{\text{taps}} - 1 \quad (2.6.6)$$

Una de las posibles ventanas a utilizar es la Hamming, definida como:

$$w[n] = 0,54 - 0,46 \cos\left(\frac{2\pi n}{N_{\text{taps}} - 1}\right), \quad 0 \leq n \leq N_{\text{taps}} - 1 \quad (2.6.7)$$

La multiplicación en el dominio del tiempo equivale a la convolución periódica de la respuesta en frecuencia ideal con el espectro de la ventana, $W(e^{j\omega})$. La ventana de Hamming tiene como características principales:

- Atenuación en banda de rechazo: Proporciona un nivel de lóbulo secundario pico de aproximadamente -41 dB y una atenuación mínima en la banda de rechazo de -53 dB.
- Suavizado del fenómeno de Gibbs: Elimina las oscilaciones de sobrepico en los bordes de la banda de paso típicas del truncamiento rectangular estricto, a cambio de un lóbulo central más ancho.

2.6.2 Transformada Rápida de Fourier

Para determinar la amplitud y fase de las señales de FI, las secuencias filtradas en el dominio del tiempo se transforman al dominio de la frecuencia mediante la Transformada Discreta de

Fourier (DFT), calculada a través del algoritmo de la Transformada Rápida de Fourier (FFT). Para una secuencia de N muestras, la expresión de análisis viene dada por [8]:

$$X[k] = \sum_{n=0}^{N-1} x[n] w[n] e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \dots, N-1 \quad (2.6.8)$$

donde $w[n]$ representa la función de la ventana aplicada a la secuencia temporal.

Dado que la captura se realiza sobre una ventana de observación finita, si la duración del intervalo de adquisición no contiene un número entero exacto de ciclos de la señal, se produce el fenómeno de fuga espectral (*spectral leakage*). Este fenómeno dispersa la energía de la frecuencia fundamental hacia los bins frecuenciales adyacentes, degradando la precisión en la estimación de amplitud y fase. Para mitigar este efecto, se aplica una función de enventanado $w[n]$. La ventana *Flattop* se caracteriza por presentar un lóbulo principal ancho y plano; esta propiedad minimiza el error de estimación de amplitud, mejorando la exactitud en la medición de niveles de tensión.

Capítulo 3

Proyecto

3.1 Descripción general del producto

Como describe la sección 2 de la Especificación de Requerimientos (??) el sistema debe generar un barrido de frecuencias en el rango de 1,8 a 2,2 GHz. La señal se debe inyectar en el DUT y se debe medir la onda reflejada. Ambas ondas (incidente y reflejada) deben bajarse a una frecuencia intermedia para luego adquirirlas y digitalizarlas con el fin de realizar el procesamiento correspondiente. Un diagrama en bloques detallado del sistema se puede observar en la figura 3.1. Los datos a asumir, requerimientos y pautas de diseño se pueden encontrar en la Especificación de Requerimientos (??).

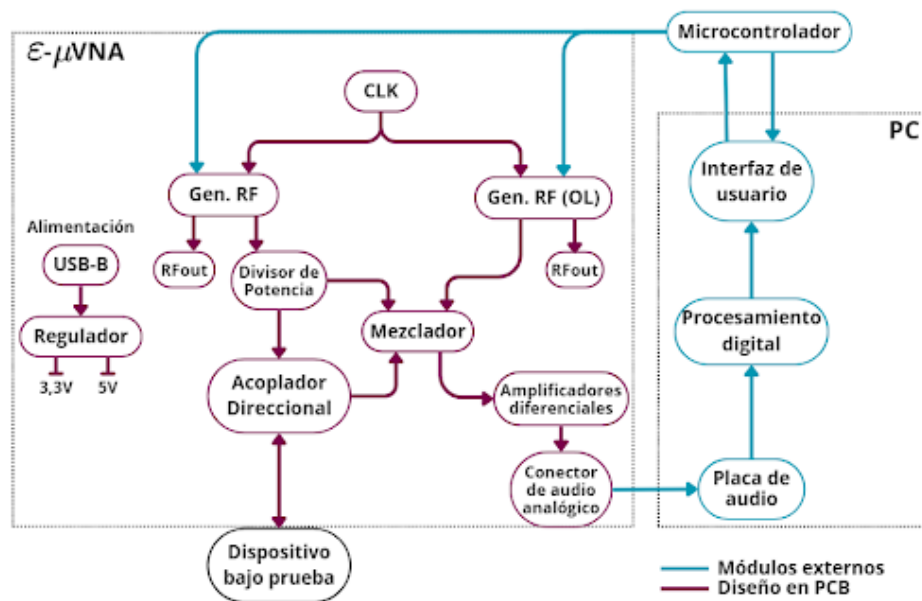


Figura 3.1: Diagrama en bloques del sistema

A continuación, se detalla la solución del hardware, software, firmware e interfaz de usuario.

3.2 Hardware

El hardware se diseñó en su totalidad en el software Altium Designer, con una licencia estudiantil gracias al programa Altium Student Lab [4]. Se creó una librería con los esquemáticos, footprints y modelos 3D de los componentes a utilizar. Se dividió el proyecto en esquemáticos que representan los distintos bloques funcionales, que se pueden observar en el capítulo "Esquemáticos

y códigos" (6.1).

3.2.1 Generadores de RF

Como explica la sección 2.1.2 de la Especificación de Requerimientos (??) se sintetizan con los circuitos integrados (IC - *Integrated Circuits*) ADF4351, un sintetizador de banda ancha con VCO integrado. Uno incide directamente en el acoplador direccional, desviando una parte hacia el mezclador para medir la señal incidente. Su esquemático se puede ver en ???. El otro se utiliza como oscilador local para el esquema heterodino (esquemático en ???). Algunas consideraciones que se tuvieron en cuenta al

pad central, filtro pll, etapa de salida (pull-up) y rf vea una unica carga
inversor fan out menos crosstalk y ringing

3.2.2 Implementación en PCB

En principio se planteó un modelo de 2 capas, con FR4 como dieléctrico de 0,8 mm de espesor.

3.3 Firmware

El firmware se programó en lenguaje C++ en el entorno Arduino IDE.

3.4 Software

3.4.1 Estimación del S11

Una vez computadas las transformadas espectrales complejas del canal de referencia, $X_{\text{ref}}[k]$, y del canal de medición, $X_{\text{med}}[k]$, un algoritmo puede proceder al cálculo del parámetro S11.

En primer lugar, se evalúa el espectro de magnitud del canal de referencia dentro de un rango frecuencial acotado alrededor de f_{FI} (banda de búsqueda del pico) para determinar el índice del bin que concentra la máxima energía:

$$k_{\text{pico}} = \arg \max_{k \in \text{Banda}} |X_{\text{ref}}[k]| \quad (3.4.1)$$

Obtenido el bin característico k_{pico} , se evalúan los valores complejos de ambas transformadas exactamente en dicha posición espectral, obteniendo los fasores complejos $V_{\text{ref}} = X_{\text{ref}}[k_{\text{pico}}]$ y $V_{\text{med}} = X_{\text{med}}[k_{\text{pico}}]$. El coeficiente de reflexión complejo crudo (no calibrado), $S_{11,\text{raw}}$, para la frecuencia de RF evaluada en el paso del barrido se calcula como el cociente vectorial complejo:

$$S_{11,\text{raw}} = \frac{V_{\text{med}}[k_{\text{pico}}]}{V_{\text{ref}}[k_{\text{pico}}]} \quad (3.4.2)$$

A partir del número complejo $S_{11,\text{raw}}$, se extraen directamente el módulo en escala logarítmica y la fase relativa:

$$|S_{11,\text{raw}}|_{\text{dB}} = 20 \log_{10} |S_{11,\text{raw}}| \quad (3.4.3)$$

$$\phi_{S_{11,\text{raw}}} = \arg(S_{11,\text{raw}}) \quad (3.4.4)$$

Este valor complejo no calibrado sirve como la entrada directa para el posterior algoritmo de corrección de errores vectoriales mediante el modelo de tres términos (SOL).

3.5 Interfaz de usuario

La interfaz de usuario se programó con PyQt6, mediante un prompt de la inteligencia artificial (IA) Claude, modelo Sonnet 5 Esfuerzo Medio. Se alimentó del software original, que tenía la lógica de medición funcionando, pero que corría secuencialmente y se comunicaba con el usuario a través de la consola.

El prompt inicial fue: "Quiero hacer una interfaz de usuario sencilla que me permita comunicarme con este script. que se pueda elegir frecuencias y paso, visualizar modulo y fase de S11 y abaco de smith, que permita hacer calibración. Qué plataforma/lenguaje recomiendas, y cuanto tiempo tardaría en ponerla en funcionamiento?". La IA recomendó Python + PyQt6/PySide6, con matplotlib embebido para los gráficos. Dió las razones (reutilizar funciones, permite actualizar el plot en vivo, aprovecha librerías de Python para graficar) y explicó otras alternativas y por qué las descartó. Preguntó si hacía un esqueleto de la app con ventana principal, campos de frecuencia, threading de barrido y gráficos, y se le dió el permiso. Creó dos archivos, un "core" que contiene la misma lógica de cálculo que el código original pero reorganizada en funciones independientes (comunicación serial con Arduino, capura de audio, procesamiento de señal, detección del tono de batido y cálculo de S11 y álgebra de calibración) y un "GUI" (Graphical User Interface) que contiene la parte visual: ventana, botones, campos y gráficos. Este llama a las funciones implementadas en el core.

Se agregó la biblioteca PyQt6 para usar ventanas, botones, campos e hilos de ejecución. Se creó una clase (SweepWorker(QThread)) que se encarga de correr el barrido en segundo plano y avisar a la ventana qué se midió para que actualice los gráficos y la barra de progreso sin congelarse entre cada medición.

La IA generó además el diagrama de flujo que se observa en la figura 3.2. Los códigos se encuentran en el capítulo "Esquemáticos y código" 6.2.1.

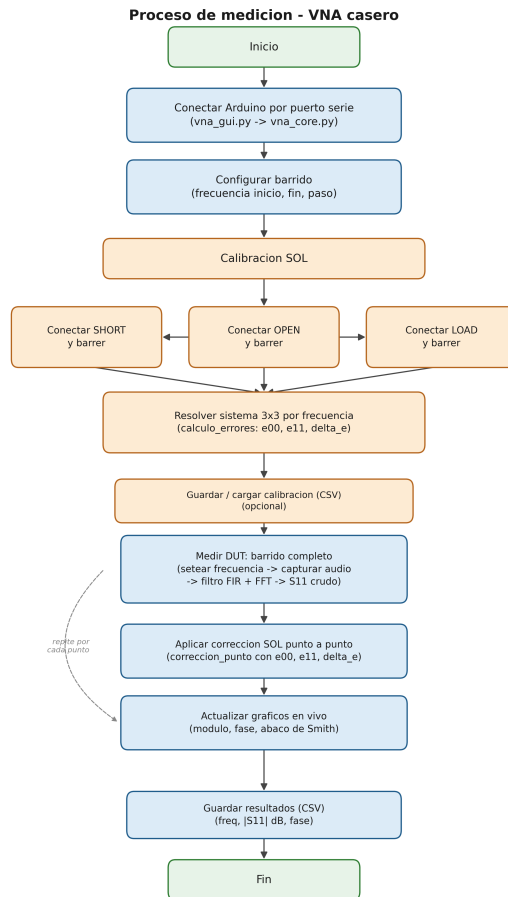


Figura 3.2: Diagrama de flujo de la interfaz de usuario

Capítulo 4

Resultados

Capítulo 5

Conclusiones y trabajos futuros

Capítulo 6

Esquemáticos y código

6.1 Esquemáticos Altium

A continuación se presentan los esquemáticos de los distintos bloques funcionales del sistema, exportados de Altium Designer.

6.2 Interfaz de usuario

6.2.1 Core

```
1  """
2  una_core.py
3  =====
4  Nucleo de adquisicion y calculo para el VNA casero (2x ADF4351 + Arduino +
5  placa de sonido). Adaptado de VNA_calibrado.py para poder ser llamado desde
6  una interfaz grafica: sin input()/tkinter bloqueantes, con callback de
7  progreso por punto y posibilidad de abortar un barrido en curso.
8
9  La logica de senales (filtro FIR, FFT, deteccion del tono de batido,
10 algebra de calibracion SOL) es la misma que la del script original; lo
11 unico que cambia es como se invoca.
12
13 Requisitos:
14     pip install pyserial sounddevice numpy scipy
15 """
16
17 import csv
18 import time
19
20 import numpy as np
21 import serial
22 import sounddevice as sd
23 from scipy.signal import firwin, lfilter
24 from scipy.signal.windows import get_window
25
26
27 # ===== Lista por defecto de frecuencias excluidas
28 ↪ =====
29 # (copiada de VNA_calibrado.py; se puede editar/reemplazar desde la GUI)
```

```

30 FRECUENCIAS_EXCLUIDAS_MHZ_DEFAULT = [
31     200.00, 204.00, 208.00, 212.00, 216.00, 217.00, 221.00, 226.00, 256.00,
    ↪ 260.00, 280.00,
32     285.00, 291.00, 320.00, 347.00, 353.00, 354.00, 359.00, 360.00, 365.00,
    ↪ 366.00, 371.00,
33     372.00, 378.00, 379.00, 385.00, 386.00, 392.00, 393.00, 400.00, 401.00,
    ↪ 408.00, 409.00,
34     416.00, 417.00, 424.00, 425.00, 433.00, 434.00, 443.00, 452.00, 512.00,
    ↪ 559.00, 560.00,
35     570.00, 571.00, 581.00, 582.00, 617.00, 640.00, 678.00, 694.00, 705.00,
    ↪ 706.00, 707.00,
36     708.00, 717.00, 718.00, 719.00, 720.00, 730.00, 731.00, 732.00, 742.00,
    ↪ 743.00, 744.00,
37     745.00, 756.00, 757.00, 758.00, 759.00, 770.00, 771.00, 772.00, 773.00,
    ↪ 784.00, 785.00,
38     786.00, 787.00, 799.00, 800.00, 801.00, 802.00, 815.00, 816.00, 817.00,
    ↪ 818.00, 831.00,
39     832.00, 833.00, 834.00, 848.00, 849.00, 850.00, 851.00, 865.00, 866.00,
    ↪ 867.00, 868.00,
40     883.00, 886.00, 904.00, 927.00, 929.00, 955.00, 1009.00, 1024.00, 1075.00,
    ↪ 1118.00, 1119.00,
41     1120.00, 1140.00, 1142.00, 1162.00, 1164.00, 1185.00, 1187.00, 1234.00,
    ↪ 1280.00, 1283.00,
42     1304.00, 1333.00, 1356.00, 1383.00, 1388.00, 1410.00, 1411.00, 1412.00,
    ↪ 1413.00, 1414.00,
43     1415.00, 1416.00, 1434.00, 1435.00, 1436.00, 1437.00, 1438.00, 1439.00,
    ↪ 1440.00, 1459.00,
44     1460.00, 1461.00, 1462.00, 1463.00, 1464.00, 1465.00, 1484.00, 1485.00,
    ↪ 1486.00, 1487.00,
45     1488.00, 1489.00, 1490.00, 1491.00, 1511.00, 1512.00, 1513.00, 1514.00,
    ↪ 1515.00, 1516.00,
46     1517.00, 1518.00, 1539.00, 1540.00, 1541.00, 1542.00, 1543.00, 1544.00,
    ↪ 1545.00, 1546.00,
47     1568.00, 1569.00, 1570.00, 1571.00, 1572.00, 1573.00, 1574.00, 1575.00,
    ↪ 1598.00, 1599.00,
48     1600.00, 1601.00, 1602.00, 1603.00, 1604.00, 1605.00, 1629.00, 1630.00,
    ↪ 1631.00, 1632.00,
49     1633.00, 1634.00, 1635.00, 1636.00, 1662.00, 1663.00, 1664.00, 1665.00,
    ↪ 1666.00, 1667.00,
50     1668.00, 1669.00, 1696.00, 1697.00, 1698.00, 1700.00, 1701.00, 1702.00,
    ↪ 1730.00, 1731.00,
51     1732.00, 1734.00, 1735.00, 1736.00, 1766.00, 1767.00, 1771.00, 1772.00,
    ↪ 1803.00, 1829.00,
52     1833.00, 1854.00, 1858.00, 1910.00, 1937.00, 2018.00, 2150.00, 2235.00,
    ↪ 2236.00,
53     2237.00, 2238.00, 2239.00, 2240.00, 2241.00, 2279.00, 2280.00, 2281.00,
    ↪ 2283.00, 2284.00,
54     2324.00, 2325.00, 2327.00, 2328.00, 2329.00, 2370.00, 2374.00, 2421.00,
    ↪ 2463.00,
55 ]
56
57

```

```

58  # ===== Configuración por defecto =====
59
60  def config_por_defecto():
61      """Devuelve un dict de configuración editable desde la GUI."""
62      return {
63          "fs_audio": 44100,
64          "duracion_captura_s": 0.6,
65          "recorte_inicial_s": 0.150,
66          "dispositivo_audio": None,
67          "f_centro_filtro": 10000,      # Hz, tono de batido esperado
68          "ancho_banda_filtro": 200,    # Hz
69          "numtaps_filtro": 1001,
70          "ventana_fft": "flattop",
71          "frecuencias_excluidas": list(FRECUENCIAS_EXCLUIDAS_MHZ_DEFAULT),
72      }
73
74
75  def banda_búsqueda_pico(config):
76      fc = config["f_centro_filtro"]
77      bw = config["ancho_banda_filtro"]
78      return (fc - bw / 2, fc + bw / 2)
79
80
81  # ===== Arduino / serial =====
82
83  class ErrorArduino(Exception):
84      pass
85
86
87  def conectar_arduino(puerto, baudrate=115200, timeout=5):
88      """Abre el puerto serie y devuelve (objeto_serial,
89      ↪ primera_linea_recibida)."""
89      ser = serial.Serial(puerto, baudrate, timeout=timeout)
90      time.sleep(2)  # el Arduino se resetea al abrir el puerto; esperamos a que
91      ↪ arranque
91      ser.reset_input_buffer()
92      linea = ser.readline().decode(errors="ignore").strip()
93      return ser, linea
94
95
96  def setear_frecuencia(ser, f_mhz):
97      """Manda la frecuencia al Arduino y espera la confirmación (OK/ERROR)."""
98      comando = f"{f_mhz:.6f}\n"
99      ser.write(comando.encode())
100     respuesta = ser.readline().decode(errors="ignore").strip()
101     intentos_vacios = 0
102     while respuesta and not respuesta.startswith("OK") and not
103     ↪ respuesta.startswith("ERROR"):
103         respuesta = ser.readline().decode(errors="ignore").strip()
104         intentos_vacios += 1
105         if intentos_vacios > 50:
106             break

```

```

107     return respuesta
108
109
110 def generar_frecuencias(inicio, fin, paso):
111     """Genera la lista de frecuencias evitando arrastre de error de punto
    ↪ flotante."""
112     if paso <= 0:
113         raise ValueError("El paso de frecuencia debe ser mayor que 0.")
114     n_pasos = round((fin - inicio) / paso)
115     return [inicio + i * paso for i in range(n_pasos + 1)]
116
117
118 # ===== Adquisicion de audio =====
119
120 def capturar_audio(duracion_s, fs, dispositivo=None, recorte_inicial_s=0.150):
121     """
122     Graba desde la entrada configurada. Devuelve (canal_ref, canal_med) como
123     arrays float64, con canal_ref = izquierdo, canal_med = derecho.
124     """
125     n_muestras = int(duracion_s * fs)
126     grabacion_con_ruido = sd.rec(
127         n_muestras, samplerate=fs, channels=2, dtype="float64",
128         ↪ device=dispositivo
129     )
130     sd.wait()
131     grabacion = grabacion_con_ruido[int(fs * recorte_inicial_s):]
132     canal_ref = grabacion[:, 0]
133     canal_med = grabacion[:, 1]
134     return canal_ref, canal_med
135
136 # ===== Filtro FIR + FFT + S11 =====
137
138 def diseñar_filtro_pasabanda(fs, f_centro, ancho_banda, numtaps,
139     ↪ duracion_captura_s):
140     n_muestras_captura = int(duracion_captura_s * fs)
141     if numtaps >= n_muestras_captura:
142         raise ValueError(
143             f"NUMTAPS_FILTRO ({numtaps}) debe ser menor que la cantidad de "
144             f"muestras por captura ({n_muestras_captura}, con
145             ↪ duracion_captura_s="
146             f"{duracion_captura_s}s y fs={fs}Hz). Bajar numtaps o subir la
147             ↪ duracion."
148         )
149     f_baja = f_centro - ancho_banda / 2
150     f_alta = f_centro + ancho_banda / 2
151     return firwin(numtaps, [f_baja, f_alta], pass_zero=False, fs=fs,
152         ↪ window="hamming")
153
154 def aplicar_filtro(señal, coeficientes):
155     return lfilter(coeficientes, 1.0, señal)

```

```

153
154
155 def calcular_espectro_complejo(señal, fs, ventana):
156     n = len(señal)
157     w = get_window(ventana, n)
158     fft_vals = np.fft.fft(señal * w)
159     fft_freqs = np.fft.fftfreq(n, d=1 / fs)
160     mitad = n // 2
161     return fft_freqs[:mitad], fft_vals[:mitad]
162
163
164 def buscar_indice_pico(freqs, espectro, banda):
165     """Devuelve el índice del bin de mayor magnitud dentro de la banda dada."""
166     f_baja, f_alta = banda
167     i0 = np.searchsorted(freqs, f_baja)
168     i1 = np.searchsorted(freqs, f_alta)
169     if i1 <= i0:
170         raise ValueError("La banda de búsqueda no contiene ningún bin de
171             ↪ frecuencia.")
172     magnitud = np.abs(espectro[i0:i1])
173     return i0 + np.argmax(magnitud)
174
175 def medir_s11_punto(canal_ref, canal_med, fs, coef_filtro, ventana_fft, banda):
176     """
177     A partir de una captura de referencia y medición, devuelve un único
178     valor complejo de S11 (sin corregir) correspondiente al tono de batido,
179     y la frecuencia de batido detectada (Hz).
180     """
181     ref_filtrada = aplicar_filtro(canal_ref, coef_filtro)
182     med_filtrada = aplicar_filtro(canal_med, coef_filtro)
183
184     freqs_ref, espectro_ref = calcular_espectro_complejo(ref_filtrada, fs,
185         ↪ ventana_fft)
186     _, espectro_med = calcular_espectro_complejo(med_filtrada, fs, ventana_fft)
187
188     idx_pico = buscar_indice_pico(freqs_ref, espectro_ref, banda)
189
190     epsilon = np.max(np.abs(espectro_ref)) * 1e-9
191     s11 = np.conj(espectro_med[idx_pico] / (espectro_ref[idx_pico] + epsilon))
192
193     return s11, freqs_ref[idx_pico]
194
195 # ===== Barrido (con callback de progreso + aborto)
196 ↪ =====
197
198 class BarridoAbortado(Exception):
199     """Se lanza cuando should_abort() devuelve True durante un barrido."""
200     pass
201

```

```

202 def barrido(ser, frecuencias_rf, coef_filtro, config, on_point=None,
↳   should_abort=None):
203     """
204     Ejecuta un barrido completo de frecuencias, midiendo un punto de S11
205     (sin corregir) por cada frecuencia.
206
207     on_point(i, n, f_rf, s11, f_batido_hz): callback opcional, llamado
208         despues de medir cada punto (i es 1-based).
209     should_abort(): callable opcional; si devuelve True se aborta el
210         barrido lanzando BarridoAbortado.
211
212     Devuelve (freqs_medidas, s11_medidos) como arrays de numpy, saltando
213     las frecuencias que esten en config["frecuencias_excluidas"].
214     """
215     resultados_freq_rf = []
216     resultados_s11 = []
217     excluidas = config.get("frecuencias_excluidas", [])
218     banda = banda_búsqueda_pico(config)
219
220     for i, f_rf in enumerate(frecuencias_rf):
221         if should_abort is not None and should_abort():
222             raise BarridoAbortado()
223
224         if any(abs(f_rf - fx) < 1e-6 for fx in excluidas):
225             continue
226
227         respuesta = setear_frecuencia(ser, f_rf)
228         if respuesta.startswith("ERROR"):
229             raise ErrorArduino(f"Arduino devolvio error en {f_rf:.3f} MHz:
↳             {respuesta}")
230
231         canal_ref, canal_med = capturar_audio(
232             config["duracion_captura_s"], config["fs_audio"],
↳             config.get("dispositivo_audio"),
233             config.get("recorte_inicial_s", 0.150),
234         )
235         s11, f_batido = medir_s11_punto(
236             canal_ref, canal_med, config["fs_audio"], coef_filtro,
237             config["ventana_fft"], banda,
238         )
239
240         resultados_freq_rf.append(f_rf)
241         resultados_s11.append(s11)
242
243         if on_point is not None:
244             on_point(i + 1, len(frecuencias_rf), f_rf, s11, f_batido)
245
246     return np.array(resultados_freq_rf), np.array(resultados_s11)
247
248
249 # ===== Calibracion SOL =====
250

```

```

251 def calculo_errores(GM_CC, GM_CA, GM_50, n):
252     """Resuelve, punto a punto, el sistema 3x3 de la calibracion SOL."""
253     e_00 = np.empty(n, dtype=complex)
254     e_11 = np.empty(n, dtype=complex)
255     delta_e = np.empty(n, dtype=complex)
256
257     for i in range(n):
258         A = np.array([
259             [1, -GM_CC[i], 1], # CC (short)
260             [1, 0, 0], # 50 (load)
261             [1, GM_CA[i], -1], # CA (open)
262         ], dtype=complex)
263         B = np.array([GM_CC[i], GM_50[i], GM_CA[i]])
264         sol = np.linalg.solve(A, B)
265         e_00[i] = sol[0]
266         e_11[i] = sol[1]
267         delta_e[i] = sol[2]
268
269     return e_00, e_11, delta_e
270
271
272 def correccion(GM, e_00, e_11, delta_e, n):
273     """Aplica la correccion de error de la calibracion SOL a un array de GM
274     ↪ medidos."""
275     Gamma = np.empty(n, dtype=complex)
276     for i in range(n):
277         denominador = (GM[i] * e_11[i] - delta_e[i])
278         if abs(denominador) < 1e-15:
279             denominador = 1e-15 + 1e-15j
280         Gamma[i] = (GM[i] - e_00[i]) / denominador
281     return Gamma
282
283 def correccion_punto(gm, e_00_i, e_11_i, delta_e_i):
284     """Version de correccion() para un unico punto (usada para graficar en
285     ↪ vivo)."""
286     denominador = gm * e_11_i - delta_e_i
287     if abs(denominador) < 1e-15:
288         denominador = 1e-15 + 1e-15j
289     return (gm - e_00_i) / denominador
290
291 def guardar_calibracion_csv(path, freqs, e_00, e_11, delta_e):
292     with open(path, "w", newline="", encoding="utf-8") as f:
293         writer = csv.writer(f)
294         writer.writerow(["freq_MHz", "e_00_real", "e_00_imag", "e_11_real",
295             ↪ "e_11_imag",
296                 "delta_e_real", "delta_e_imag"])
297         for i in range(len(freqs)):
298             writer.writerow([
299                 f"{freqs[i]:.6f}",
300                 f"{e_00[i].real:.10e}", f"{e_00[i].imag:.10e}",

```

```

300         f"{e_11[i].real:.10e}", f"{e_11[i].imag:.10e}",
301         f"{delta_e[i].real:.10e}", f"{delta_e[i].imag:.10e}",
302     ])
303
304
305 def cargar_calibracion_csv(path):
306     """Rearma los valores complejos guardados por guardar_calibracion_csv()."""
307     freqs, e_00, e_11, delta_e = [], [], [], []
308     with open(path, "r", encoding="utf-8") as f:
309         reader = csv.DictReader(f)
310         for row in reader:
311             freqs.append(float(row["freq_MHz"]))
312             e_00.append(complex(float(row["e_00_real"]),
313                               ↪ float(row["e_00_imag"])))
313             e_11.append(complex(float(row["e_11_real"]),
314                               ↪ float(row["e_11_imag"])))
314             delta_e.append(complex(float(row["delta_e_real"]),
315                               ↪ float(row["delta_e_imag"])))
315     return np.array(freqs), np.array(e_00), np.array(e_11), np.array(delta_e)
316
317
318 def guardar_resultados_csv(path, freq_rf, modulo_db, fase_deg):
319     with open(path, "w", newline="", encoding="utf-8") as f:
320         writer = csv.writer(f)
321         writer.writerow(["freq_rf_MHz", "modulo_S11_dB", "fase_S11_deg"])
322         for f_rf, m, fa in zip(freq_rf, modulo_db, fase_deg):
323             writer.writerow([f"{f_rf:.6f}", f"{m:.4f}", f"{fa:.4f}"])
324
325
326 def modulo_fase(s11_array):
327     """Convierte un array de S11 complejos a (modulo_dB, fase_grados)."""
328     modulo_db = 20 * np.log10(np.abs(s11_array) + 1e-12)
329     fase_deg = np.degrees(np.angle(s11_array))
330     return modulo_db, fase_deg

```

6.2.2 Graphical User Interface

```

1  """
2  vna_gui.py
3  =====
4  Interfaz grafica (PyQt6) para el VNA casero: permite elegir frecuencia de
5  inicio/fin/paso, conectar el Arduino, correr la calibracion SOL (short /
6  open / load) paso a paso, medir un DUT, y visualizar en vivo el modulo y
7  la fase de S11 junto con el abaco de Smith.
8
9  Requisitos:
10     pip install PyQt6 pyserial sounddevice numpy scipy matplotlib
11
12  Ejecutar con:
13     python vna_gui.py
14  """
15
16  import os

```

```

17 import sys
18
19 import numpy as np
20
21 from PyQt6.QtCore import Qt, QThread, pyqtSignal
22 from PyQt6.QtWidgets import (
23     QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
24     QGroupBox, QLabel, QLineEdit, QPushButton, QDoubleSpinBox,
25     QProgressBar, QFileDialog, QMessageBox, QPlainTextEdit, QSplitter,
26 )
27
28 from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas
29 from matplotlib.figure import Figure
30
31 import vna_core as core
32
33
34 # ===== Hilo de trabajo: corre un barrido sin trabar la GUI
35 ↪ =====
36
37 class SweepWorker(QThread):
38     """Ejecuta core.barrido() en un hilo aparte y reenvia sus resultados como
39     ↪ señales Qt."""
40
41     punto_medido = pyqtSignal(int, int, float, complex, float) # i, n, f_rf,
42     ↪ s11, f_batido
43     terminado = pyqtSignal(object, object) # freqs
44     ↪ (ndarray), s11 (ndarray)
45     abortado = pyqtSignal()
46     error = pyqtSignal(str)
47
48     def __init__(self, ser, frecuencias_rf, coef_filtro, config, parent=None):
49         super().__init__(parent)
50         self.ser = ser
51         self.frecuencias_rf = frecuencias_rf
52         self.coef_filtro = coef_filtro
53         self.config = config
54         self._abortar = False
55
56     def abortar(self):
57         self._abortar = True
58
59     def run(self):
60         try:
61             freqs, s11 = core.barrido(
62                 self.ser, self.frecuencias_rf, self.coef_filtro, self.config,
63                 on_point=lambda i, n, f, s, fb: self.punto_medido.emit(i, n, f,
64                     ↪ s, fb),
65                 should_abort=lambda: self._abortar,
66             )
67             self.terminado.emit(freqs, s11)
68         except core.BarridoAbortado:

```

```

64         self.abortado.emit()
65     except Exception as e: # noqa: BLE001 - se muestra en la GUI
66         self.error.emit(str(e))
67
68
69 # ===== Dibujo del abaco de Smith (fondo estatico)
70 ↪ =====
71
72 def dibujar_fondo_smith(ax):
73     """Dibuja el grillado clasico de un abaco de Smith (r y x constantes)."""
74     ax.clear()
75     ax.set_aspect("equal")
76     ax.axis("off")
77     ax.set_xlim(-1.15, 1.15)
78     ax.set_ylim(-1.15, 1.15)
79     ax.set_title("Abaco de Smith")
80
81     theta = np.linspace(0, 2 * np.pi, 400)
82
83     # Circulo unitario (|Gamma| = 1)
84     ax.plot(np.cos(theta), np.sin(theta), color="black", linewidth=1.0)
85     ax.axhline(0, color="gray", linewidth=0.5)
86
87     # Circulos de resistencia constante (r = 0, 0.2, 0.5, 1, 2, 5)
88     for r in (0.0, 0.2, 0.5, 1.0, 2.0, 5.0):
89         cx = r / (r + 1)
90         rad = 1 / (r + 1)
91         x = cx + rad * np.cos(theta)
92         y = rad * np.sin(theta)
93         ax.plot(x, y, color="gray", linewidth=0.5)
94
95     # Arcos de reactancia constante (x = +/- 0.2, 0.5, 1, 2, 5)
96     t = np.linspace(0, 2 * np.pi, 800)
97     for x_val in (0.2, 0.5, 1.0, 2.0, 5.0):
98         for signo in (1, -1):
99             xv = signo * x_val
100             cx, cy = 1.0, 1.0 / xv
101             rad = abs(1.0 / xv)
102             gx = cx + rad * np.cos(t)
103             gy = cy + rad * np.sin(t)
104             mask = gx ** 2 + gy ** 2 <= 1.0 + 1e-6
105             ax.plot(gx[mask], gy[mask], color="gray", linewidth=0.5)
106
107 # ===== Ventana principal =====
108
109 class MainWindow(QMainWindow):
110     def __init__(self):
111         super().__init__()
112         self.setWindowTitle("VNA - Control y medición de S11")
113         self.resize(1200, 750)
114

```

```

115         self.config = core.config_por_defecto()
116         self.ser = None
117         self.hilo = None
118
119         # Estado de calibracion
120         self.cal_paso = 0          # 0 = sin empezar, 1=SHORT, 2=OPEN, 3=LOAD,
            ↪ 4=lista
121         self.cal_gm_cc = None
122         self.cal_gm_ca = None
123         self.cal_gm_50 = None
124         self.cal_freqs = None
125         self.cal_e00 = None
126         self.cal_e11 = None
127         self.cal_delta_e = None
128         self.cal_disponible = False
129
130         # Buffers de la medicion en curso (para graficar en vivo)
131         self.buf_freq = []
132         self.buf_mod_db = []
133         self.buf_fase_deg = []
134         self.buf_gamma = []
135         self.ultima_medicion_freqs = None
136         self.ultima_medicion_gamma = None
137         self.ultima_medicion_mod_db = None
138         self.ultima_medicion_fase_deg = None
139
140         self._armar_ui()
141
142         # ----- UI -----
143
144     def _armar_ui(self):
145         central = QWidget()
146         self.setCentralWidget(central)
147         layout_principal = QHBoxLayout(central)
148
149         splitter = QSplitter(Qt.Orientation.Horizontal)
150         layout_principal.addWidget(splitter)
151
152         panel_izq = self._armar_panel_controles()
153         panel_der = self._armar_panel_graficos()
154
155         splitter.addWidget(panel_izq)
156         splitter.addWidget(panel_der)
157         splitter.setStretchFactor(0, 0)
158         splitter.setStretchFactor(1, 1)
159
160     def _armar_panel_controles(self):
161         panel = QWidget()
162         panel.setMinimumWidth(340)
163         panel.setMaximumWidth(420)
164         v = QVBoxLayout(panel)
165

```

```

166     # --- Conexion Arduino ---
167     gb_conexion = QGroupBox("Conexion Arduino")
168     f = QVBoxLayout(gb_conexion)
169     fila_puerto = QHBoxLayout()
170     fila_puerto.addWidget(QLabel("Puerto:"))
171     self.ed_puerto = QLineEdit("COM3")
172     fila_puerto.addWidget(self.ed_puerto)
173     f.addLayout(fila_puerto)
174
175     fila_baud = QHBoxLayout()
176     fila_baud.addWidget(QLabel("Baudrate:"))
177     self.ed_baudrate = QLineEdit("115200")
178     fila_baud.addWidget(self.ed_baudrate)
179     f.addLayout(fila_baud)
180
181     self.btn_conectar = QPushButton("Conectar")
182     self.btn_conectar.clicked.connect(self._conectar_arduino)
183     f.addWidget(self.btn_conectar)
184
185     self.lbl_estado_conexion = QLabel("Desconectado")
186     f.addWidget(self.lbl_estado_conexion)
187     v.addWidget(gb_conexion)
188
189     # --- Barrido ---
190     gb_barrido = QGroupBox("Barrido de frecuencia")
191     f2 = QVBoxLayout(gb_barrido)
192
193     self.sp_f_inicio = self._nuevo_spin(1800.0, 0, 6000, " MHz")
194     self.sp_f_fin = self._nuevo_spin(2200.0, 0, 6000, " MHz")
195     self.sp_f_paso = self._nuevo_spin(1.0, 0.001, 1000, " MHz")
196
197     f2.addWidget(self._fila("F inicio:", self.sp_f_inicio))
198     f2.addWidget(self._fila("F fin:", self.sp_f_fin))
199     f2.addWidget(self._fila("F paso:", self.sp_f_paso))
200     v.addWidget(gb_barrido)
201
202     # --- Calibracion SOL ---
203     gb_cal = QGroupBox("Calibracion SOL")
204     f3 = QVBoxLayout(gb_cal)
205
206     self.lbl_estado_cal = QLabel("Sin calibrar")
207     f3.addWidget(self.lbl_estado_cal)
208
209     self.btn_cal_iniciar = QPushButton("Iniciar calibracion nueva")
210     self.btn_cal_iniciar.clicked.connect(self._iniciar_calibracion)
211     f3.addWidget(self.btn_cal_iniciar)
212
213     self.btn_cal_paso = QPushButton("Barrer paso actual")
214     self.btn_cal_paso.setEnabled(False)
215     self.btn_cal_paso.clicked.connect(self._barrer_paso_calibracion)
216     f3.addWidget(self.btn_cal_paso)
217

```

```

218     self.btn_cal_cargar = QPushButton("Cargar calibracion desde archivo...")
219     self.btn_cal_cargar.clicked.connect(self._cargar_calibracion)
220     f3.addWidget(self.btn_cal_cargar)
221
222     self.btn_cal_guardar = QPushButton("Guardar calibracion actual...")
223     self.btn_cal_guardar.setEnabled(False)
224     self.btn_cal_guardar.clicked.connect(self._guardar_calibracion)
225     f3.addWidget(self.btn_cal_guardar)
226
227     v.addWidget.gb_cal)
228
229     # --- Medicion DUT ---
230     gb_dut = QGroupBox("Medición del DUT")
231     f4 = QVBoxLayout(gb_dut)
232
233     self.btn_medir = QPushButton("Medir DUT")
234     self.btn_medir.setEnabled(False)
235     self.btn_medir.clicked.connect(self._iniciar_medicion_dut)
236     f4.addWidget(self.btn_medir)
237
238     self.btn_abortar = QPushButton("Abortar barrido")
239     self.btn_abortar.setEnabled(False)
240     self.btn_abortar.clicked.connect(self._abortar_barrido)
241     f4.addWidget(self.btn_abortar)
242
243     self.progreso = QProgressBar()
244     f4.addWidget(self.progreso)
245
246     self.btn_guardar_resultados = QPushButton("Guardar resultados CSV...")
247     self.btn_guardar_resultados.setEnabled(False)
248     self.btn_guardar_resultados.clicked.connect(self._guardar_resultados)
249     f4.addWidget(self.btn_guardar_resultados)
250
251     v.addWidget(gb_dut)
252
253     # --- Log ---
254     gb_log = QGroupBox("Log")
255     f5 = QVBoxLayout(gb_log)
256     self.txt_log = QTextEdit()
257     self.txt_log.setReadOnly(True)
258     f5.addWidget(self.txt_log)
259     v.addWidget(gb_log, stretch=1)
260
261     return panel
262
263     def _armar_panel_graficos(self):
264         panel = QWidget()
265         v = QVBoxLayout(panel)
266
267         # Figura módulo + fase (dos subplots apilados)
268         self.fig_mf = Figure(figsize=(6, 5), facecolor="#fad7ea")
269         self.ax_mod = self.fig_mf.add_subplot(211)

```

```

270     self.ax_fase = self.fig_mf.add_subplot(212, sharex=self.ax_mod)
271     self._preparar_ejes_mod_fase()
272     self.canvas_mf = FigureCanvas(self.fig_mf)
273
274     # Figura abaco de Smith
275     self.fig_smith = Figure(figsize=(5, 5), facecolor="#fad7ea")
276     self.ax_smith = self.fig_smith.add_subplot(111)
277     dibujar_fondo_smith(self.ax_smith)
278     (self.linea_smith,) = self.ax_smith.plot([], [], color="deeppink",
279                                             marker=".", linestyle="None")
280     self.canvas_smith = FigureCanvas(self.fig_smith)
281
282     fila_graficos = QHBoxLayout()
283     fila_graficos.addWidget(self.canvas_mf, stretch=1)
284     fila_graficos.addWidget(self.canvas_smith, stretch=1)
285     v.addLayout(fila_graficos)
286
287     return panel
288
289 def _preparar_ejes_mod_fase(self):
290     self.ax_mod.clear()
291     self.ax_mod.set_facecolor("#fad7ea")
292     self.ax_mod.set_title("S11 - Módulo")
293     self.ax_mod.set_ylabel("|S11| (dB)")
294     self.ax_mod.grid(True, alpha=0.3)
295     (self.linea_mod,) = self.ax_mod.plot([], [], color="deeppink",
296                                         marker=".", linestyle="None")
297
298     self.ax_fase.clear()
299     self.ax_fase.set_facecolor("#fad7ea")
300     self.ax_fase.set_title("S11 - Fase")
301     self.ax_fase.set_xlabel("Frecuencia de RF (MHz)")
302     self.ax_fase.set_ylabel("Fase (grados)")
303     self.ax_fase.grid(True, alpha=0.3)
304     (self.linea_fase,) = self.ax_fase.plot([], [], color="darkmagenta",
305                                         marker=".", linestyle="None")
306     self.fig_mf.tight_layout()
307
308 @staticmethod
309 def _nuevo_spin(valor, minimo, maximo, sufijo):
310     sp = QDoubleSpinBox()
311     sp.setDecimals(3)
312     sp.setRange(minimo, maximo)
313     sp.setValue(valor)
314     sp.setSuffix(sufijo)
315     return sp
316
317 @staticmethod
318 def _fila(etiqueta, widget):
319     w = QWidget()
320     h = QHBoxLayout(w)
321     h.setContentsMargins(0, 0, 0, 0)

```

```

322         h.addWidget(QLabel(etiqueta))
323         h.addWidget(widget)
324         return w
325
326     def _log(self, texto):
327         self.txt_log.appendPlainText(texto)
328
329     # ----- Conexion --
330
331     def _conectar_arduino(self):
332         puerto = self.ed_puerto.text().strip()
333         try:
334             baudrate = int(self.ed_baudrate.text().strip())
335         except ValueError:
336             QMessageBox.warning(self, "Baudrate invalido", "El baudrate debe ser
337                 ↪ un numero entero.")
338             return
339
340         try:
341             self.ser, linea = core.conectar_arduino(puerto, baudrate)
342         except Exception as e:
343             QMessageBox.critical(self, "Error de conexion", str(e))
344             self.lbl_estado_conexion.setText("Error de conexion")
345             return
346
347         self.lbl_estado_conexion.setText(f"Conectado ({puerto} @ {baudrate})")
348         if linea:
349             self._log(f"Arduino: {linea}")
350             self._log(f"Conectado a {puerto} @ {baudrate} baud.")
351
352     # ----- Utilidades --
353
354     def _construir_frecuencias(self):
355         inicio = self.sp_f_inicio.value()
356         fin = self.sp_f_fin.value()
357         paso = self.sp_f_paso.value()
358         return core.generar_frecuencias(inicio, fin, paso)
359
360     def _construir_filtro(self):
361         return core.diseñar_filtro_pasabanda(
362             self.config["fs_audio"], self.config["f_centro_filtro"],
363             self.config["ancho_banda_filtro"], self.config["numtaps_filtro"],
364             self.config["duracion_captura_s"],
365         )
366
367     def _validar_listo_para_barrer(self):
368         if self.ser is None:
369             QMessageBox.warning(self, "Sin conexion", "Primero conecta el
370                 ↪ Arduino.")
371             return False
372         if self.hilo is not None and self.hilo.isRunning():

```

```

371         QMessageBox.warning(self, "Barrido en curso", "Ya hay un barrido
        ↪     corriendo.")
372         return False
373     return True
374
375 def _lanzar_barrido(self, frecuencias, on_terminado, on_abortado=None,
    ↪     on_error=None):
376     coef_filtro = self._construir_filtro()
377     self.hilo = SweepWorker(self.ser, frecuencias, coef_filtro, self.config)
378     self.hilo.punto_medido.connect(self._on_punto_generico)
379     self.hilo.terminado.connect(on_terminado)
380     self.hilo.abortado.connect(on_abortado or self._on_abortado_generico)
381     self.hilo.error.connect(on_error or self._on_error_generico)
382     self.progreso.setValue(0)
383     self.btn_abortar.setEnabled(True)
384     self.hilo.start()
385
386 def _on_punto_generico(self, i, n, f_rf, s11, f_batido):
387     self.progreso.setMaximum(n)
388     self.progreso.setValue(i)
389     mod_db = 20 * np.log10(abs(s11) + 1e-12)
390     fase_deg = np.degrees(np.angle(s11))
391     self._log(f"[{i}/{n}] RF={f_rf:.3f} MHz batido={f_batido:.1f} Hz "
392             f"|S11|={mod_db:.2f} dB fase={fase_deg:.1f} deg")
393
394 def _on_abortado_generico(self):
395     self.btn_abortar.setEnabled(False)
396     self._log("Barrido abortado por el usuario.")
397
398 def _on_error_generico(self, mensaje):
399     self.btn_abortar.setEnabled(False)
400     QMessageBox.critical(self, "Error durante el barrido", mensaje)
401     self._log(f"ERROR: {mensaje}")
402
403 def _abortar_barrido(self):
404     if self.hilo is not None and self.hilo.isRunning():
405         self.hilo.abortar()
406
407 # ----- Calibracion -----
408
409 ETIQUETAS_PASO_CAL = {
410     1: "Paso 1/3: conecte SHORT (cortocircuito) y presione 'Barrer paso
        ↪     actual'.",
411     2: "Paso 2/3: conecte OPEN (circuito abierto) y presione 'Barrer paso
        ↪     actual'.",
412     3: "Paso 3/3: conecte LOAD (50 ohm) y presione 'Barrer paso actual'.",
413 }
414
415 def _iniciar_calibracion(self):
416     if not self._validar_listo_para_barrer():
417         return
418     self.cal_paso = 1

```

```

419         self.cal_gm_cc = self.cal_gm_ca = self.cal_gm_50 = None
420         self.cal_disponible = False
421         self.btn_medir.setEnabled(False)
422         self.btn_cal_guardar.setEnabled(False)
423         self.btn_cal_paso.setEnabled(True)
424         self.lbl_estado_cal.setText(self.ETIQUETAS_PASO_CAL[1])
425
426     def _barrer_paso_calibracion(self):
427         if not self._validar_listo_para_barrer():
428             return
429         frecuencias = self._construir_frecuencias()
430         self.btn_cal_paso.setEnabled(False)
431         self.btn_cal_iniciar.setEnabled(False)
432         self._lanzar_barrido(
433             frecuencias,
434             on_terminado=self._on_paso_calibracion_terminado,
435             on_abortado=self._on_calibracion_abortada,
436         )
437
438     def _on_calibracion_abortada(self):
439         self._on_abortado_generico()
440         self.btn_cal_paso.setEnabled(True)
441         self.btn_cal_iniciar.setEnabled(True)
442
443     def _on_paso_calibracion_terminado(self, freqs, gm):
444         self.btn_abortar.setEnabled(False)
445         self.btn_cal_iniciar.setEnabled(True)
446         self.cal_freqs = freqs
447
448         if self.cal_paso == 1:
449             self.cal_gm_cc = gm
450             self.cal_paso = 2
451             self.lbl_estado_cal.setText(self.ETIQUETAS_PASO_CAL[2])
452             self.btn_cal_paso.setEnabled(True)
453         elif self.cal_paso == 2:
454             self.cal_gm_ca = gm
455             self.cal_paso = 3
456             self.lbl_estado_cal.setText(self.ETIQUETAS_PASO_CAL[3])
457             self.btn_cal_paso.setEnabled(True)
458         elif self.cal_paso == 3:
459             self.cal_gm_50 = gm
460             self._finalizar_calculo_calibracion()
461
462     def _finalizar_calculo_calibracion(self):
463         n = len(self.cal_freqs)
464         if not (len(self.cal_gm_cc) == len(self.cal_gm_ca) ==
465             ↪ len(self.cal_gm_50) == n):
466             QMessageBox.warning(
467                 self, "Inconsistencia",
468                 "Los tres barridos de calibracion (SHORT/OPEN/LOAD) no tienen la
469                 ↪ "

```

```

468         "misma cantidad de puntos. Revisa que no se haya abortado
469         ↪ ninguno "
470         "a mitad de camino."
471     )
472     self.cal_paso = 0
473     self.btn_cal_paso.setEnabled(False)
474     return
475
476     self.cal_e00, self.cal_e11, self.cal_delta_e = core.calculo_errores(
477         self.cal_gm_cc, self.cal_gm_ca, self.cal_gm_50, n
478     )
479     self.cal_disponible = True
480     self.cal_paso = 4
481     self.lbl_estado_cal.setText(f"Calibracion lista ({n} puntos).")
482     self.btn_cal_paso.setEnabled(False)
483     self.btn_cal_guardar.setEnabled(True)
484     self.btn_medir.setEnabled(True)
485     self._log("Calibracion SOL calculada correctamente.")
486
487     def _cargar_calibracion(self):
488         path, _ = QFileDialog.getOpenFileName(
489             self, "Cargar calibracion", "", "CSV (*.csv)"
490         )
491         if not path:
492             return
493         try:
494             freqs, e00, e11, delta_e = core.cargar_calibracion_csv(path)
495         except Exception as e:
496             QMessageBox.critical(self, "Error al cargar", str(e))
497             return
498
499         self.cal_freqs, self.cal_e00, self.cal_e11, self.cal_delta_e = freqs,
500         ↪ e00, e11, delta_e
501         self.cal_disponible = True
502         self.cal_paso = 4
503         self.lbl_estado_cal.setText(f"Calibracion cargada desde archivo
504         ↪ ({len(freqs)} puntos).")
505         self.btn_medir.setEnabled(True)
506         self.btn_cal_guardar.setEnabled(False)
507         self._log(f"Calibracion cargada de {path}")
508
509     def _guardar_calibracion(self):
510         if not self.cal_disponible:
511             return
512         path, _ = QFileDialog.getSaveFileName(
513             self, "Guardar calibracion", "calibracion.csv", "CSV (*.csv)"
514         )
515         if not path:
516             return
517         core.guardar_calibracion_csv(path, self.cal_freqs, self.cal_e00,
518         ↪ self.cal_e11, self.cal_delta_e)
519         self._log(f"Calibracion guardada en {path}")

```

```

516
517 # ----- Medicion DUT -----
518
519 def _iniciar_medicion_dut(self):
520     if not self._validar_listo_para_barrer():
521         return
522     if not self.cal_disponible:
523         QMessageBox.warning(self, "Sin calibracion", "Primero completa o
524             ↪ carga una calibracion.")
525         return
526
527     frecuencias = self._construir_frecuencias()
528
529     # Reiniciar buffers y graficos
530     self.buf_freq, self.buf_mod_db, self.buf_fase_deg, self.buf_gamma = [],
531         ↪ [], [], []
532     self._preparar_ejes_mod_fase()
533     self.canvas_mf.draw()
534     dibujar_fondo_smith(self.ax_smith)
535     (self.linea_smith,) = self.ax_smith.plot([], [], color="deeppink",
536         ↪ marker=".", linestyle="None")
537     self.canvas_smith.draw()
538
539     self.btn_medir.setEnabled(False)
540     self.btn_guardar_resultados.setEnabled(False)
541
542     self._lanzar_barrido(
543         frecuencias,
544         on_terminado=self._on_medicion_dut_terminada,
545         on_abortado=self._on_medicion_dut_abortada,
546     )
547     # Reconectamos punto_medido a un manejador especifico para DUT
548     # (ademas del log generico que ya se conecto en _lanzar_barrido)
549     self.hilo.punto_medido.connect(self._on_punto_dut)
550
551 def _on_medicion_dut_abortada(self):
552     self._on_abortado_generico()
553     self.btn_medir.setEnabled(True)
554
555 def _indice_calibracion_valido(self, i_1_based):
556     idx = i_1_based - 1
557     return self.cal_e00 is not None and 0 <= idx < len(self.cal_e00)
558
559 def _on_punto_dut(self, i, n, f_rf, s11_crudo, f_batido):
560     # Si hay calibracion punto-a-punto disponible, corregimos en vivo.
561     if self._indice_calibracion_valido(i):
562         idx = i - 1
563         gamma = core.correccion_punto(
564             s11_crudo, self.cal_e00[idx], self.cal_e11[idx],
565             ↪ self.cal_delta_e[idx]
566         )
567     else:

```

```

564         gamma = s11_crudo # fallback: se corrige todo al final
565
566     mod_db = 20 * np.log10(abs(gamma) + 1e-12)
567     fase_deg = np.degrees(np.angle(gamma))
568
569     self.buf_freq.append(f_rf)
570     self.buf_mod_db.append(mod_db)
571     self.buf_fase_deg.append(fase_deg)
572     self.buf_gamma.append(gamma)
573
574     self.linea_mod.set_data(self.buf_freq, self.buf_mod_db)
575     self.linea_fase.set_data(self.buf_freq, self.buf_fase_deg)
576     for ax in (self.ax_mod, self.ax_fase):
577         ax.relim()
578         ax.autoscale_view()
579     self.canvas_mf.draw_idle()
580
581     self.linea_smith.set_data([g.real for g in self.buf_gamma],
582                               [g.imag for g in self.buf_gamma])
583     self.canvas_smith.draw_idle()
584
585     def _on_medicion_dut_terminada(self, freqs, gm_dut):
586         self.btn_abortar.setEnabled(False)
587         self.btn_medir.setEnabled(True)
588
589         if len(freqs) != len(self.cal_e00):
590             QMessageBox.warning(
591                 self, "Advertencia de consistencia",
592                 "La cantidad de puntos medidos no coincide con la calibracion "
593                 "(revisa el rango de frecuencias). Se muestran los valores "
594                 "corregidos en vivo, pero puede haber un corrimiento de indice."
595             )
596
597         n = min(len(freqs), len(self.cal_e00))
598         gamma_corregido = core.correccion(
599             gm_dut[:n], self.cal_e00[:n], self.cal_e11[:n],
600             ↪ self.cal_delta_e[:n], n
601         )
602         mod_db, fase_deg = core.modulo_fase(gamma_corregido)
603
604         self.ultima_medicion_freqs = freqs[:n]
605         self.ultima_medicion_gamma = gamma_corregido
606         self.ultima_medicion_mod_db = mod_db
607         self.ultima_medicion_fase_deg = fase_deg
608
609         self.btn_guardar_resultados.setEnabled(True)
610         self._log(f"Medicion de DUT finalizada: {n} puntos corregidos.")
611
612     def _guardar_resultados(self):
613         if self.ultima_medicion_freqs is None:
614             return
615         path, _ = QFileDialog.getSaveFileName(

```

```

615         self, "Guardar resultados", "resultado_S11.csv", "CSV (*.csv)"
616     )
617     if not path:
618         return
619     core.guardar_resultados_csv(
620         path, self.ultima_medicion_freqs,
621         self.ultima_medicion_mod_db, self.ultima_medicion_fase_deg,
622     )
623     self._log(f"Resultados guardados en {path}")
624
625     # ----- Cierre -----
626
627     def closeEvent(self, event):
628         if self.hilo is not None and self.hilo.isRunning():
629             self.hilo.abortar()
630             self.hilo.wait(2000)
631         if self.ser is not None:
632             try:
633                 self.ser.close()
634             except Exception:
635                 pass
636         event.accept()
637
638
639     def main():
640         app = QApplication(sys.argv)
641         app.setStyleSheet("""
642             QMainWindow, QWidget, QGroupBox {background-color: #fad7ea; }""")
643         ventana = MainWindow()
644         ventana.show()
645         sys.exit(app.exec())
646
647
648     if __name__ == "__main__":
649         main()

```

Apéndice

Bibliografía

- [1] Sitio Web Oficial. URL: <https://www.freestylelibre.com.ar/>.
- [2] Altium. *Consideraciones para el diseño de PCB de alta velocidad: Prácticas recomendadas para la colocación y el enrutamiento de condensadores de desacoplo*. Dec. 2020. URL: <https://resources.altium.com/es/p/high-speed-pcb-design-considerations-bypass-capacitor-place-and-route-tips>.
- [3] Altium. *Via Stitching and Via Shielding in PCB Design*. s. f. URL: <https://www.altium.com/documentation/altium-designer/via-stitching-via-shielding-pcb>.
- [4] Altium Limited. *Altium Designer*. Version 26.1.1. Licencia educativa. 2026. URL: <https://www.altium.com>.
- [5] Analog Devices. *HMC601 75 dB logarithmic detector/controller*. <https://www.analog.com/en/products/hmc601.html>. n.d.
- [6] Tektronix Inc. *Vector Network Analysis: A Practical Introduction*. 2023. URL: https://download.tek.com/document/70W_60918_0_Tek_VNA_PR1.pdf?_gl=1*wh717q*_gcl_au*MjQyMjEyMTE5LjE3ODk0ODkzMjU.*_ga*NjkwMjAxNzcwLjE3ODk0ODkzMjU.*_ga_1HMYs1JH9M*cze30TAxODY4MjkkbzIkZzAkDDE30TAxODY4MzIkajU3JGwwJGgwJGRFXzVmVFRzOGMwbmFqaG8zX
- [7] Keysight Technologies. *778D Coaxial Dual-Directional Coupler, 100 MHz to 2 GHz*. s. f. URL: <https://www.keysight.com/us/en/product/778D/coaxial-dualdirectional-coupler-100-mhz-to-2-ghz.html>.
- [8] Dimitris G. Manolakis and Vinay K. Ingle. *Applied Digital Signal Processing: Theory and Practice*. Cambridge, UK: Cambridge University Press, 2011. ISBN: 978-0-521-11002-0.
- [9] Mini-Circuits. *ZABDC20-322H-S+ Coaxial Directional Coupler*. s. f. URL: <https://www.minicircuits.com/WebStore/dashboard.html?model=ZABDC20-322H-S%2B>.
- [10] Alan V. Oppenheim and Ronald W. Schaffer. *Discrete-time signal processing, second edition*. Upper Saddle River, NJ: Prentice Hall, 1999. ISBN: 0-13-754920-2.
- [11] World Health Organization. *Diabetes*. Sitio Web Oficial. Nov. 2024. URL: <https://www.who.int/news-room/fact-sheets/detail/diabetes>.
- [12] World Health Organization. *Global report on diabetes*. Sitio Web Oficial. 2016. URL: <https://www.who.int/publications/i/item/9789241565257>.
- [13] Hewlett Packard Inc. Rytting D. “Network analyzer error models and calibration methods”. In: *HP* (1998).
- [14] Urvashi Sengal. *Directional Couplers: Their Operation and Application*. Mini-Circuits. Oct. 2020. URL: <https://blog.minicircuits.com/directional-couplers-their-operation-and-application/>.
- [15] E. Topsakal, T. Karacolak, and E. C. Moreland. “Glucose-dependent dielectric properties of blood plasma.” In: *Proceedings of the XXXth URSI General Assembly and Scientific Symposium* (2011). DOI: 10.1109/URSIGASS.2011.6051324.